Lec5. The Rise of Reasoning Models

2026.10. · Post-Training Course (RLHF Book Ch.7, Nathan Lambert)

Table of Contents

Part1. From RL to Reasoning

Part2. The Reasoning Model Cambrian Explosion

Part3. Common Implementation Patterns

Part4. Paper Review: MiniMax-M1

- Lightning attention · FP32 LM head · CISPO

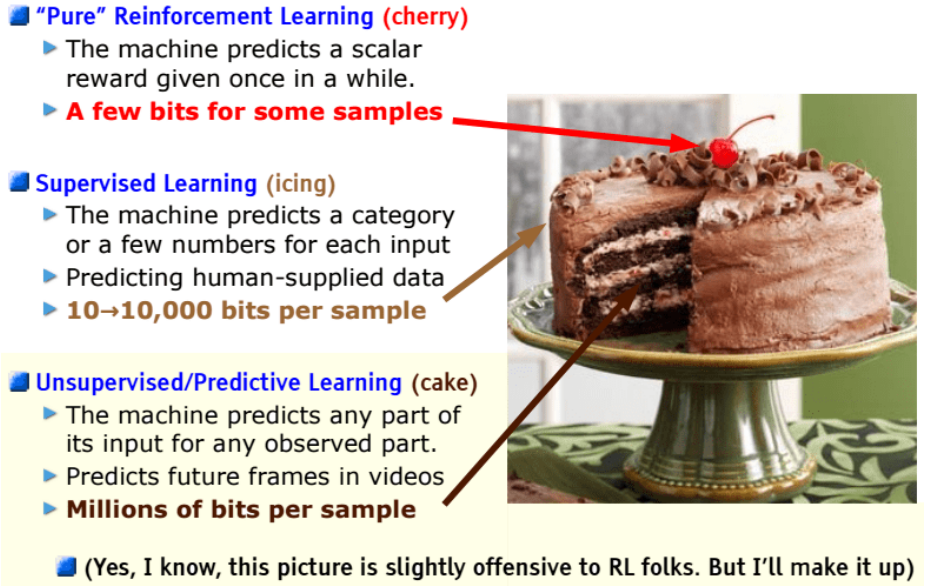
Take-Home Message & Open Questions

Part1. From RL to Reasoning

Lec3-4: PPO, GRPO, loss aggregation, async training (RL 구현)
Lec5: reasoning model의 등장 → 2025년 핵심 모델 → 공통 recipe

The LeCun Cake (Recall)

- NeurIPS 2016, Yann LeCun의 cake 비유
  - Self-supervised learning (internet pretraining) = **cake 본체**
  - Supervised fine-tuning / mid-training = **icing**
  - Reinforcement learning (RLHF → RLVR) = cherry on top
- Language model에서 이 비유가 그대로 실현됨
  - RL은 세상에서 많은 정보를 모으지만, 탄탄한 foundation 위에서만 작동
- Agent 시대: RL compute가 pretraining을 넘어설 수도 있음
  - 그래도 우리가 거쳐온 **학습 loop의 층위**를 설명하는 비유로는 여전히 유효



Synced, 2019

RLHF vs RLVR — reward가 바뀌면 모든 게 바뀐다

RLHF — subjective scoring

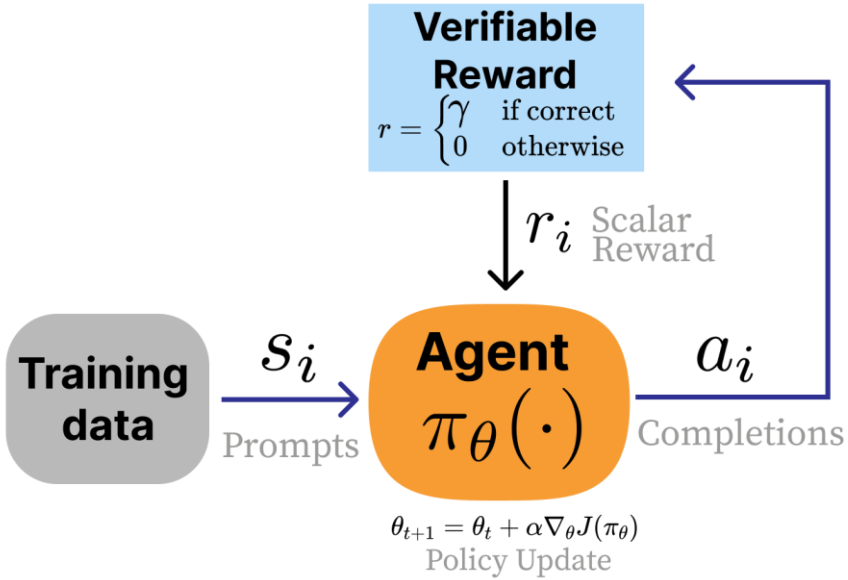
```
Q. Explain opportunity cost in economics.
```

- clarity · accuracy · completeness 판단 = 정답 없는 **learned preference**
- 학습된 reward model → 쉽게 over-optimize (style hacking)

RLVR — verifiable scoring

```
Q. What is the sum of all primes < 20?
   extracted_answer == 77  →  Reward = 1
Q. Write fib(n) returning the nth Fibonacci number.
   assert fib(10) == 55  →  all tests pass  →  Reward = 1
```

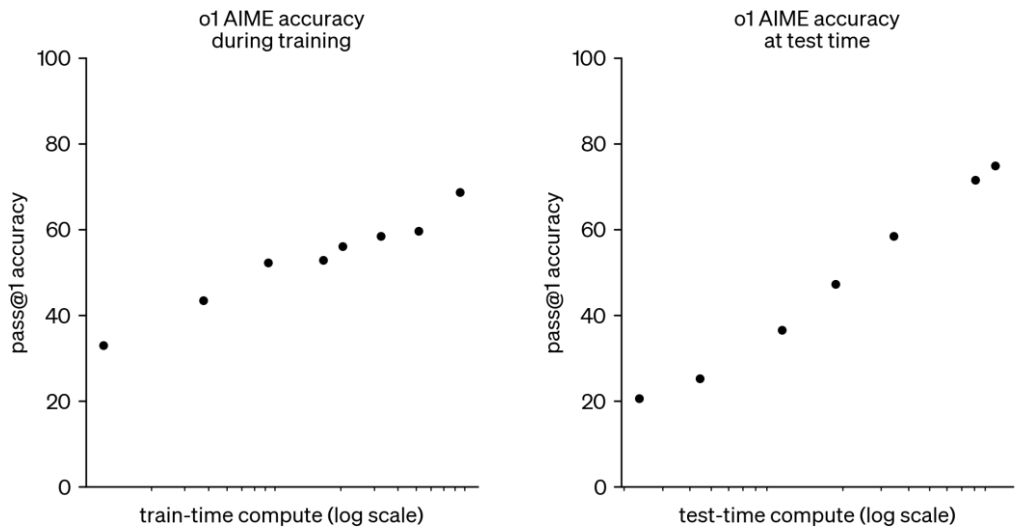
→ 정확하고 정밀한 신호 → 훨씬 더 멀리 최적화 가능



RLVR feedback loop: reward model 대신 verifier가 completion을 채점. Tool-use RL에선 이 verifier가 다시 environment가 되는 중 (Lambert et al., 2024)

알고리즘(PPO, GRPO)은 그대로, 바뀐 건 reward source → "정답을 이미 아는 문제로 RL을 scale할 수 있다"는 새 직관이 reasoning model의 문을 엶. 실제 post-training은 RLHF + RLVR + rubric을 섞어 쓰는 messy한 과정

Train-time vs Test-time Scaling & Thinking Tokens



OpenAI, 2024 — 좌: train-time compute / 우: test-time compute (x축 log scale)

- Train-time: RL run을 길게 할수록 pass@1 ↑ (log-linear)
- Test-time: 더 많은 token으로 생각할수록 성능 ↑ = **inference-time scaling**
- 두 축은 얽혀 있음 — 초기 reasoning 시대 = 큰 RL run이 inference-time scaling을 unlock

What thinking tokens look like

Standard model — 바로 답함

The primes less than 20 are 2, 3, 5, 7, 11, 13, 17, 19. Their sum is 77.

Reasoning model — <think> token을 먼저 생성

```
<think> I need to find all primes less than 20. 2 is
prime. 3 is prime. 4 = 2×2, not prime. 5 is prime… Let me
add them up: 2+3=5, 5+5=10, 10+7=17, … 58+19=77. </think>
The answer is \boxed{77}.
```

- Verification: `extracted_answer == 77` → Reward = 1
- 어려운 문제는 thinking이 수천 token — 반복적이고 guess & check 같은 "이상한" 형태
- open-weight 모델은 trace 공개, closed 모델은 숨김

Key Terms for This Lecture

Evaluation metrics

- **pass@1**: 문제당 sample 1개를 뽑았을 때의 정확도
- **pass@K**: K개 생성 중 하나라도 맞으면 정답 → **coverage** 측정
- n번 시도 (n ≥ K) 중 c개가 정답일 때 조합으로 추정 (unbiased):

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$



- n = 8, c = 2 → pass@1 = 2/8 = 0.25
- pass@4 = 1 − C(6,4)/C(8,4) = 1 − 15/70 ≈ **0.79**
- pass@K − pass@1 gap = test-time scaling·RL이 노릴 headroom

Algorithm terms

DAPO (Yu et al., 2025) — Decoupled Clip and Dynamic sAmpling PO

- clip-higher (ε_high > ε_low)로 exploration 유지
- dynamic sampling: all-correct / all-wrong group 제외
- token-level loss, overlong reward shaping
- open data + 강한 RL-Zero math 성능 → 많은 후속 논문이 인용

CISPO (MiniMax-M1, 2025) — Clipped IS-weight PO

- per-token ratio 대신 importance sampling weight를 clip
- 초기 논문이지만 2026에도 relevance 계속 증가
- → Part4 paper review에서 자세히

The Research Before o1 & Why RL Works Now

The research that came before

Work	Idea
STaR / Quiet-STaR (2022–24)	CoT를 sample → 정답을 낸 rationale만 keep → 학습 (ground-truth reward)
TRICE (2023)	MCMC 기반 reasoning trace 최적화 — 복잡한 알고리즘은 scale이 어려움
VinePPO (2024)	prompt당 많은 rollout + Monte Carlo 추정으로 correctness 기반 credit (GSM8K/MATH)
Tülu 3 (2024)	PPO로 math correctness 학습 + broad capability 유지 → 'RLVR' 용어 탄생

→ 아이디어 자체는 새롭지 않음. o1/R1 이후 단순한 방식으로 scale한 것이 차이 (simplicity scales)

Why does RL work now?

1. **Infra & stability**: RL tooling·async infra가 수년간 open에서 성숙 (entropy collapse, long-horizon credit은 여전히 연구 과제)
2. **Base model이 충분히 좋아짐**: mid-training에 고품질 math/code 추가 → ~2024 이후 모델부터 viable (2025.1 연구 표준 = Qwen 2.5)
3. **Verifiable domain = clean signal**: math/code는 train ↔ eval이 일치 → AlpacaEval·MT-Bench·Arena류의 gaming 문제 회피

Part2. The Reasoning Model Cambrian Explosion

o1 (2024.9) 이후 2025년 한 해 동안 쏟아진 모델들
지금 돌아보면 어떤 모델이 field의 궤적을 바꿨나?

The Reasoning Model Cambrian Explosion



OpenAI o1
Sep 12, 2024



DeepSeek R1
Jan 20, 2025



Kimi 1.5
Jan 20, 2025



Open-Reasoner-Zero
Mar 31, 2025



Qwen 3
Apr 29, 2025



Phi-4 Reasoning
Apr 30, 2025



MiMo
Apr 30, 2025



Skywork OR-1
May 2025



INTELLECT-2
May 12, 2025



OpenThoughts 3
Jun 5, 2025



Magistral
Jun 10, 2025



MiniMax-M1
Jun 16, 2025



Kimi K2
Jul 11, 2025



GLM-4.5
Jul 28, 2025



OLMo 3 Think
Nov 20, 2025



DeepSeek V3.2
Dec 1, 2025



Nemotron 3 Nano
Dec 15, 2025



MiMo-V2-Flash
Dec 16, 2025

+ many more
25+ models in 2025

o1 이후 수개월 lag → 2025년 'wall of models'. 지금 돌아보면 궤적을 바꾼 모델과 잊혀질 모델이 구분됨

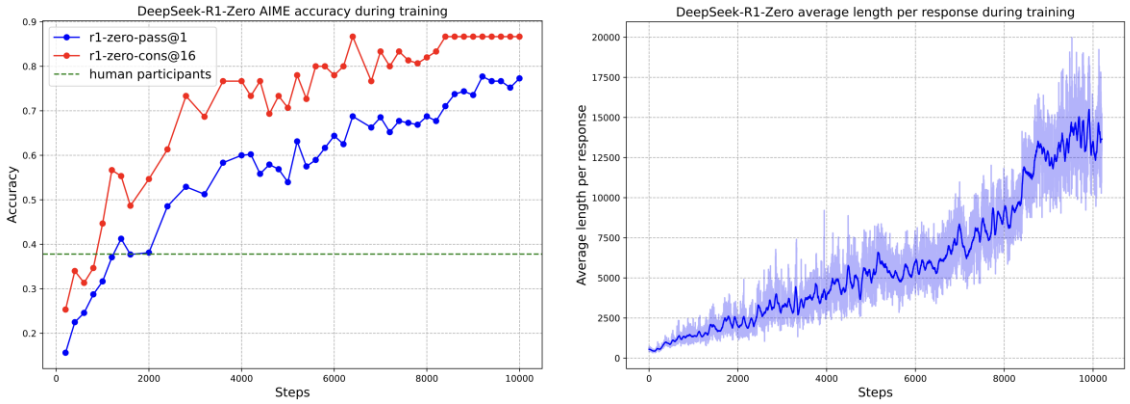
DeepSeek R1 & Kimi 1.5 (Jan. 20, 2025) — 같은 날 나온 두 report

DeepSeek R1 — The catalyst

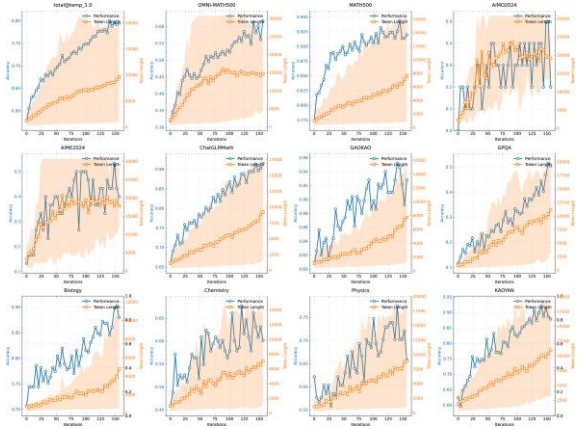
- o1 이후 놀랍게 빠른 replication + 전체 recipe 공개 → 누구나 할 수 있는 기술임을 보여줌
- **R1-Zero**: base model에 SFT 없이 pure RL → 응답 길이 ↑와 함께 CoT reasoning이 emergent
- Full recipe: cold-start SFT → large-scale RL → small model distillation
- 2025.12 on-policy distillation 전까지 사실상 default recipe

Kimi 1.5 — Scaling the curriculum

- R1과 같은 날 공개, open weight가 없어 상대적으로 묻힘
- difficulty scheduling + online filtering → 유용한 gradient 유지
- progressive length extension → overthinking ↓, long CoT 가능
- PPO/GRPO-style RL — 대부분 아이디어가 이후 recipe로 생존



R1-Zero training curve (Guo et al., 2025)



accuracy와 response length가 함께 증가 (Team et al., 2025)

Open Replications & Foundations (Feb.–Jun. 2025)

Open-Reasoner-Zero (Feb. 18)

- 첫 minimal public replication: code, curated RL data(129k), weights 전부 공개
- Vanilla PPO + GAE (λ=1, γ=1), rule-based reward, no KL
- GRPO 아닌 PPO로도 충분 → "Don't sleep on PPO": multi-turn tool use에선 value model의 per-token credit이 다시 중요해질 수 있음

Qwen 3 (Apr. 29)

- Llama 4 flop 직후 새 open-weight 연구 표준
- /think · /no_think toggleable thinking + thinking budget → 대부분 lab이 training headache로 결론, 2026엔 reasoning 모델 하나 + low effort mode
- 소형 모델은 strong-to-weak distillation (off-policy → on-policy logit matching)

MiMo (Apr. 30, Xiaomi 7B)

- pretraining → post-training 전 pipeline 공개 (25T tokens 3-stage mixing, MTP)
- Base · SFT · RL **중간 checkpoint**를 공개한 드문 open reasoning model
- multi-domain RL로 단일 task over-optimization 방지

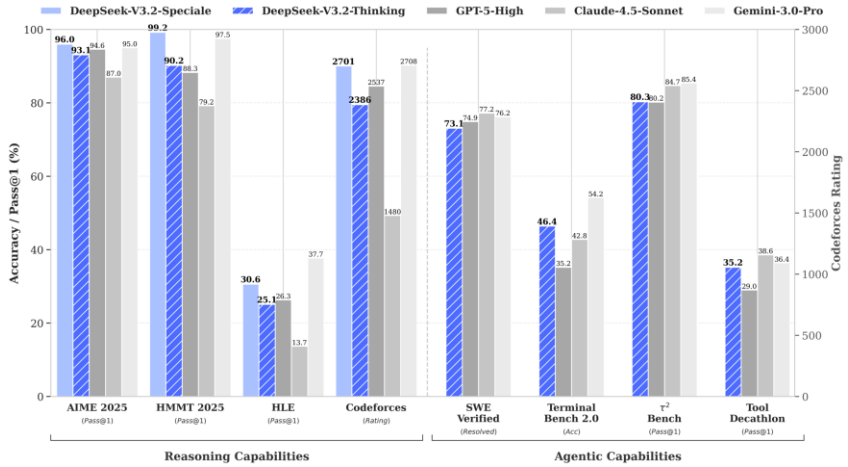
OpenThoughts 3 (Jun. 5)

- 1.2M examples (math, code, science), QwQ-32B traces, 1,000+ data pipeline 실험
- OpenThinker3-7B: **SFT only**로 강한 reasoning
- 1년이 지나도 이를 처음부터 대체한 open SFT dataset이 없음 (Olmo 3 Dolci SFT가 확장판)

Reasoning → Agentic (Jul.–Dec. 2025)

Model	Date	핵심
GLM-4.5	Jul. 28	355B / 32B active MoE · thinking + direct mode · expert-model iteration + RL (agent, reasoning, chat). 일찍부터 **agentic** messaging, 중국 open-weight 리더십을 굳힌 모델
Olmo 3 Think	Nov. 20	stage · checkpoint · data · infra · hyperparameter 전부 공개한 **fully open** reasoning model. Olmo 3.1: ~200 GPU로 약 1달 multi-domain RL — fully open 중 가장 긴 RL run
DeepSeek V3.2	Dec. 1	reasoning을 V3 line에 통합 (별도 reasoning model X) · thinking을 **tool use**에 직접 통합 · Speciale variant = max reasoning
Nemotron 3 Nano	Dec. 15	31.6B / ~3.2B active, hybrid Mamba-Transformer MoE (long context 메모리 효율) · SFT + multi-env RLVR + RLHF · open data · env · checkpoint → NVIDIA의 AI2-like 행보 시작

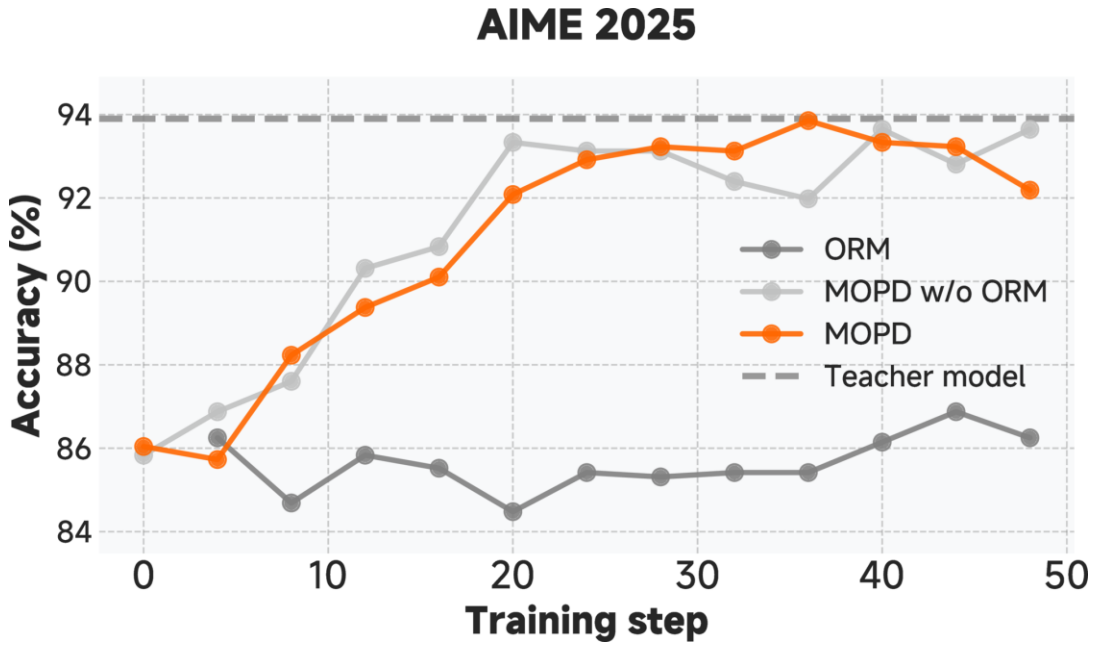
- 2025 하반기: competition math/code reasoning → tool use & agents가 진짜 frontier (OpenAI o3가 game changer)
- agent 전환이 늦은 모델은 더 도드라짐 — Lambert 본인도 Olmo 3가 math · code · IF 중심이라 이 흐름에서 늦었다고 회고
- reasoning 전용 모델 → thinking이 기본 모델과 tool use에 녹아드는 방향 (DeepSeek V3.2)



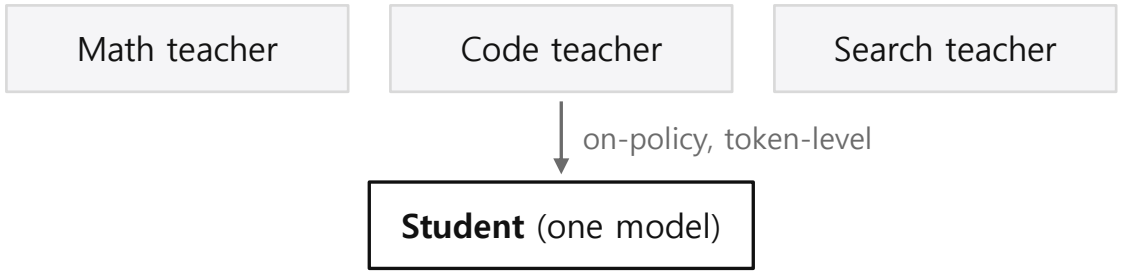
DeepSeek V3.2: reasoning + agentic benchmark (DeepSeek-AI, 2025)

MiMo-V2-Flash (Dec. 16, 2025) — Multi-Teacher OPD

- 309B / 15B active MoE (fast rollout용), OpenRouter 무료 hosting으로 사용량 급증
- 핵심은 post-training: domain별 **specialist teacher** (math, code, search, reasoning …) 학습 → 하나의 student로 distill
- Student는 sequence-level outcome reward가 아니라 teacher의 dense, token-level reward로 학습 → sample efficient, 모든 domain에서 strongest teacher 수준
- 조직 관점: teacher별로 팀을 나눠 병렬 개발 → 한 팀이 통합 recipe 담당 → post-training org를 scale
- 이후 DeepSeek V4, Nemotron 3 Ultra 등이 채택 → 새로운 post-training 표준



MOPD: domain teacher의 dense token-level reward가 student를 teacher 수준으로, sequence-level ORM은 정체 (LLM-Core Xiaomi, 2025)



2026 Open Models & What the Landscape Tells Us

2026 open models

Model	핵심
Nemotron 3 Super (3/10)	120B/12B MoE, LatentMoE, MTP, NVFP4 · fully open
Arcee Trinity-Large-Thinking (4/1)	long-horizon multi-turn tool use · Apache 2.0 (US)
Kimi K2.5 → K2.6 (1/27 → 4/20)	native multimodal, agent swarm → long-horizon coding
GLM-5 → 5.1 (2/12 → 4/7)	agentic engineering → 길고 안정적인 autonomous run
DeepSeek V4 Preview (4/24)	Pro / Flash, dual reasoning mode, 1M context

frontier: "reasoning model" → long-horizon, tool-using agent substrate

What the landscape tells us

1. RL basics는 open-source에서 빠르게 replicate — pretraining과 달리 big lab에 silo되지 않음
2. math/code reasoning → agentic 전환이 매우 빨랐음
3. long-context inference 효율을 위한 architecture 혁신 (특히 compute가 부족한 중국 lab)
4. Lasting fundamentals (FP32 LM head, CISPO-style loss)는 일찍 나왔지만, noise 속에서 당시엔 알아보기 어려웠음 (→ Part4)
5. 모든 모델이 공유하는 trick은 드묾 → model building은 path-dependent, **항상 직접 실험**

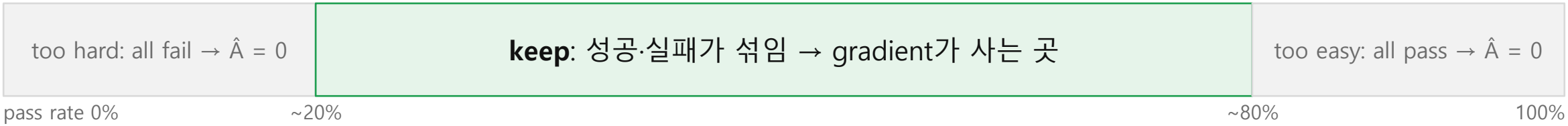
Part3. Common Implementation Patterns

여러 모델이 (전부는 아니지만) 공통으로 택한 RL recipe 결정들
→ 이어서 Part4. Paper review: MiniMax-M1

Recipe ① Data: Difficulty Filtering & Active Sampling

GRPO advantage는 group 내 reward에 분산이 있을 때만 0이 아님 → 전부 맞거나 전부 틀린 prompt는 gradient 0 = compute 낭비

$$\hat{A}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^{G})}{\text{std}(\{R_j\}_{j=1}^{G})}$$



- **Offline difficulty filtering**: 학습 전 초기 policy (SFT 후 or base)로 prompt당 N개 sample → pass rate ~20–80%만 남김. 가장 널리 채택된 trick
  - *Seed-Thinking 1.5, Open-Reasoner-Zero, Phi-4 Reasoning, INTELLECT-2, MiMo, Skywork OR-1, Olmo 3*
- **Online filtering & curriculum**: 학습 중 이미 100% 맞히는 prompt는 skip, 어려운 문제는 뒤로 저장
  - *Kimi 1.5, Magistral, Llama-Nemotron, INTELLECT-2, MiMo*
- **Zero-gradient filtering + active sampling**: Â = 0인 group을 버리고 nonzero-gradient prompt로 batch를 다시 채움 → loss가 매우 안정 (pretraining처럼 LR schedule이 필요해 보일 정도) — *Olmo 3*

Recipe ② Loss: KL, Clipping, Normalization

KL penalty — dropped, now creeping back

- RLHF: over-optimization (style 붕괴) 방지용 regularizer
- single-turn math RLVR: verifiable reward는 hack이 어려움 → **β = 0** (Magistral, ORZ, Skywork OR-1)
- 긴 off-policy agentic RL: 모델이 멀리 drift하지 않게 KL/reference term 부활 (e.g. Cursor Composer 2)

$$\max_{\theta} \mathbb{E}\left[R(x,y)\right] - \beta \mathcal{D}_{KL}\left(\pi_{\theta} \,\|\, \pi_{ref}\right)$$

Relaxed / asymmetric clipping (DAPO clip-higher)

- upper bound만 넓혀 확률 낮은 token의 exploratory update를 살림 (Magistral, INTELLECT-2 등 DAPO 계열)
- ε 값 하나가 asymptotic 성능 자체를 바꿈 (ScaleRL) → CISPO의 동기 (Part4)

$$\text{clip}\left(r_{i,t},\ 1-\epsilon_{low},\ 1+\epsilon_{high}\right), \quad \epsilon_{low}=0.2,\ \epsilon_{high}=0.28$$

Normalization — 평균 내는 방식이 objective를 바꾼다

- Advantage: prompt group이 아닌 **batch 단위** normalize → 소수의 쉬운/어려운 prompt가 지배하지 않게 (Magistral, MiMo)
- Loss: sequence 평균이 아닌 **token 평균** → length bias 제거 (Dr. GRPO, DAPO) — Lec4 참고

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\ell_{i,t}$$

per-sequence (GRPO): 긴 응답일수록 token당 weight ↓

$$\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\ell_{i,t}$$

per-token (DAPO, Dr. GRPO): 모든 token 동일 weight

Recipe ③ Reward Shaping & Length Control

$$R(x, y) = R_{correct} + \lambda_f R_{format} + \lambda_l R_{lang} - \lambda_{len} P_{len}$$

- **Format reward**: 올바른 <think>…</think> 구조, 추출 가능한 answer에 reward → correctness가 아니라 usability (inference engine 이 parse 가능해야 함) — R1, ORZ, Magistral, Skywork OR-1
- **Language consistency reward**: trace 도중 중국어·프랑스어로 switching → 불안정, 사용성 ↓ → 언어 일관성에 reward — R1, Magistral
- **Length control**: 긴 context가 유리하지만, 정답에 필요한 만큼만 짧게 생각하는 모델이 목표
  - Progressive length extension (Kimi 1.5)
  - Small length penalty (INTELLECT-2), overlong filtering (throughput)
  - 아직 tech report에 잘 정리되지 않은 열린 연구 영역

Progressive length extension

Stage 1 · short max length

Stage 2 · longer

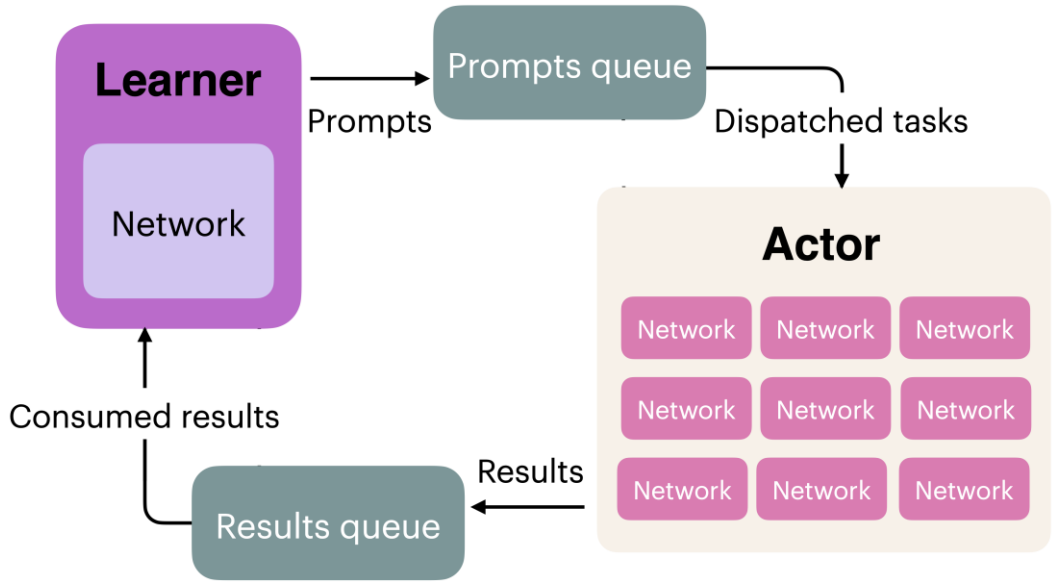
Stage 3 · full context

max response length (token budget) →

짧은 budget에서 배울 수 있는 걸 다 배운 뒤 budget을 늘림 → **concise**하면서도 long CoT 가능한, downstream 에서 효율적인 모델

Recipe ④ Infrastructure & Findings

- **Inference is the bottleneck**: 길고 길이가 제각각인 completion → Olmo 3: learner GPU ~75% idle, inference compute가 training의 5–14×
- **Off-policy / async updates**: actor는 계속 생성, learner는 소비 → rollout이 약간 stale해지지만 throughput 급증 (Seed-Thinking 1.5, INTELLECT-2, Olmo 3)
  - 대가: π_old(생성) ≠ π_θ(학습) → IS correction과 train/infer mismatch가 중요 → FP32 LM head, CISPO
- **Test-time scaling**: majority voting (R1, Phi-4), selector model (DeepSeek-GRM) — pass@K vs pass@1 gap = headroom



Distributed (async) RL training infrastructure (Olmo et al., 2025)

Findings

- text-only reasoning RL이 multimodal(vision) 성능도 boost — 좁은 domain post-training이 generalize (MiMo-VL, Magistral)
- **midtraining이 RL ceiling을 결정**: pretraining의 math/code 비중이 RL이 도달할 수 있는 지점을 bound (MiMo)

Part4. Paper Review: MiniMax-M1

Scaling Test-Time Compute Efficiently with Lightning Attention (MiniMax, 2025.6)

① Architecture: Lightning attention ② Precision: FP32 LM head ③ Algorithm: CISPO

MiniMax-M1 한눈에 (Overview)

문제의식: test-time compute를 늘릴수록 성능 ↑ — 그런데 softmax attention은 길이 n에 대해 $O(n^2)$
→ 긴 reasoning과 긴 RL rollout이 너무 비쌈. 목표: 긴 생성을 효율적으로 scale하는 open-weight reasoning model

- 456B total / 45.9B active MoE (32 experts)
- 1M input / 80K output tokens
- 40K · 80K thinking budget 두 버전 open-weight

MiniMax-Text-01
base model

→

Continual pretraining
reasoning 중심 7.5T tokens

→

SFT
long CoT · reflection 패턴 주입

→

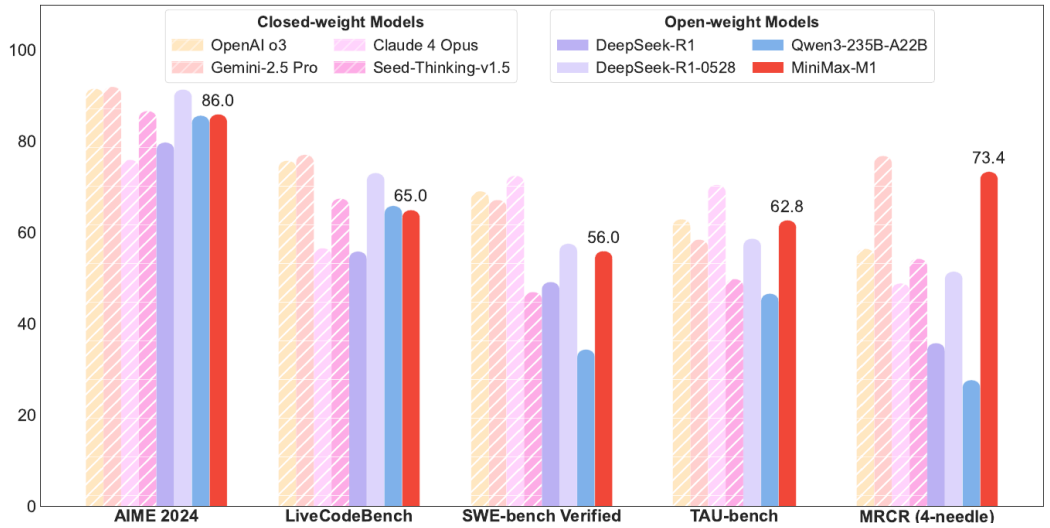
Large-scale RL
512 × H800, 3주 ≈ $0.53M

세 가지 기여는 하나의 흐름

① **Lightning attention**
긴 생성을 싸게 (100K에서 R1 대비 FLOPs ~25%)

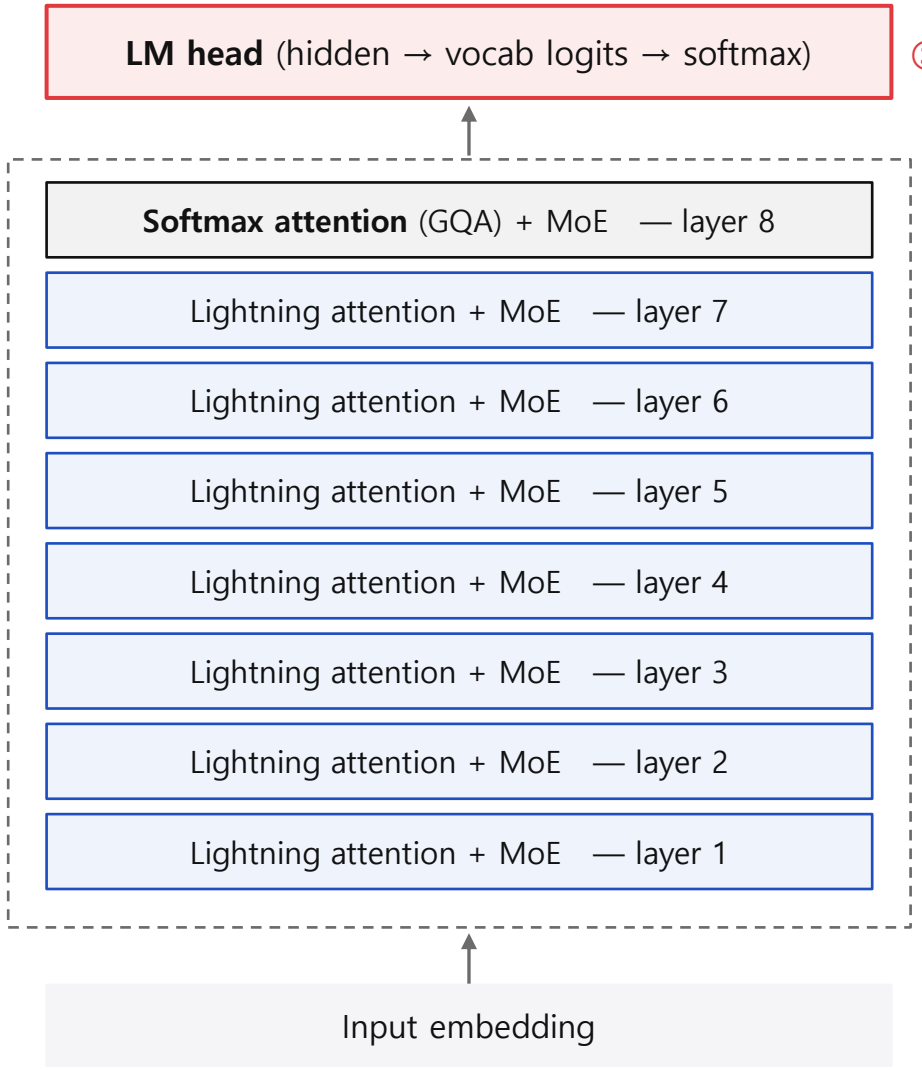
② **FP32 LM head**
그 구조에서 터진 학습/추론 확률 불일치를 해결

③ **CISPO**
남은 RL 효율 문제를 알고리즘으로 (DAPO 대비 2×)

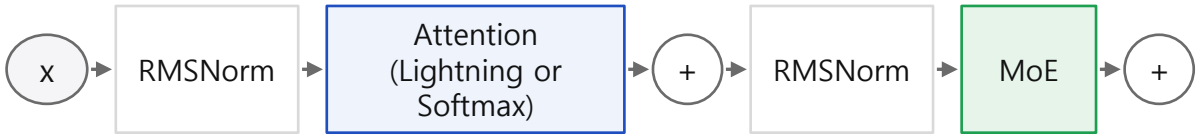


M1-80K (빨강): SWE-bench 56.0 · TAU-bench 62.8 (> Gemini 2.5 Pro) · MRCR 73.4 (> o3, Claude 4 Opus) — MiniMax-M1 Fig.1

전체 아키텍처: Hybrid Attention + MoE (MiniMax-Text-01 기반)



Block 하나 (Transformer-style)



residual 연결에 DeepNorm 스케일(α) 적용 · Lightning block: Q·K·V에 SiLU, 출력에 sigmoid gate + RMSNorm

MoE: token마다 32개 expert 중 2개만

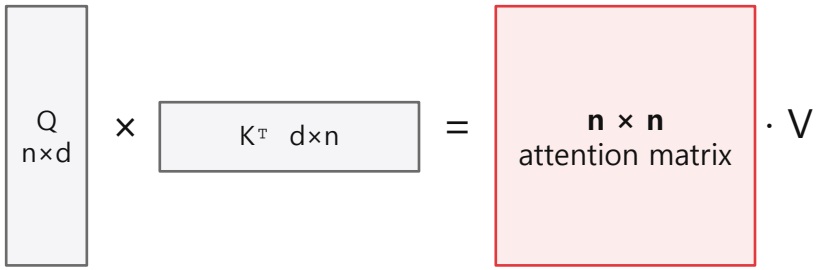


항목	값
hidden size / layers	6144 / 80 (7 lightning : 1 softmax)
attention heads	64 heads × 128 dim, softmax 층은 GQA (group 8)
position	RoPE를 head dim의 절반에만 (base 10,000)
MoE	32 experts, top-2, expert FFN 9216, global router
context	1M tokens (R1의 8배)

MiniMax-01 (2025) Sec.2, MiniMax-M1 (2025) Sec.1 기준

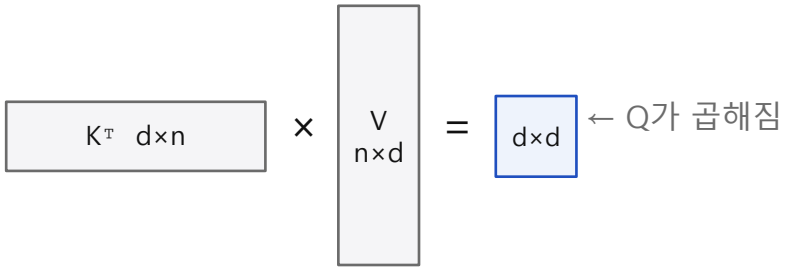
① Lightning Attention: 긴 생성을 싸게

Softmax attention

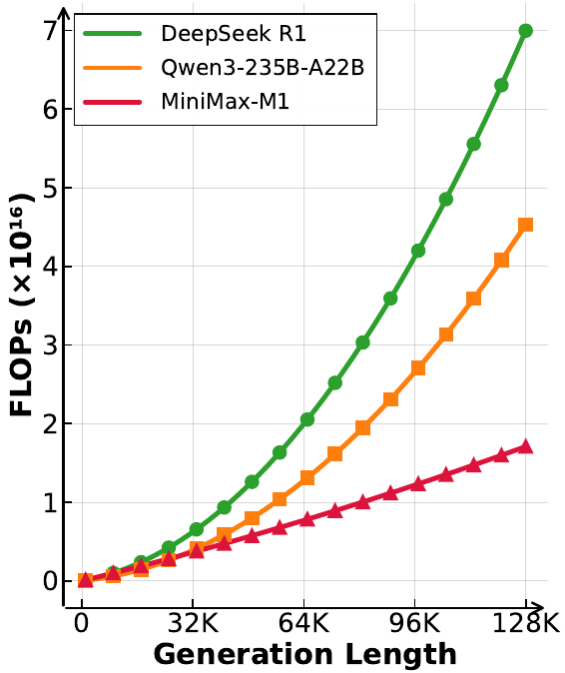


계산 $O(n^2d)$ — 길이가 2배면 4배
생성 시 KV cache가 길이에 비례해 계속 커짐

Linear attention (right product trick)



계산 $O(nd^2)$ — 길이에 선형 ($d \ll n$)
생성 시 d×d state만 갱신: $kv_t = kv_{(t-1)} + k_t v_t^T$



생성 길이별 이론 FLOPs: 64K에서 R1 대비 50% 미만, 100K에서 약 25% (MiniMax-M1 Fig.1)

- **Lightning** = linear attention의 I/O-aware 구현. causal mask 때문에 생기는 느린 cumsum을 tiling으로 회피 (block 내부는 left product, block 사이는 right product)
- **Hybrid 7 : 1** — 정확한 retrieval이 필요한 경우를 위해 softmax 층을 8층마다 하나 유지

RL과의 연결: RL의 병목은 생성 (Recipe ④) → 긴 생성이 싸지면 RL을 더 길게, 더 많이 돌릴 수 있음 → M1 전체 RL ≈ $0.53M

② FP32 LM Head: 학습-추론 확률 불일치

(1) 같은 token의 확률을 두 프로그램이 따로 계산

	하는 일	예시
Inference engine	rollout 생성, 뽑은 token 확률 기록 → **p_infer**	vLLM, SGLang
Trainer	같은 token 확률 재계산 → **p_train** → gradient	Megatron, HF

같은 weight · 같은 token → 이론상 p_infer = p_train. 그런데 kernel이 달라 실제로는 다름: 연산 순서·합치기, bf16 반올림, 특히 LM head의 큰 logit을 softmax(exp)가 증폭

(2) IS ratio는 update 전이면 정확히 1이어야 함

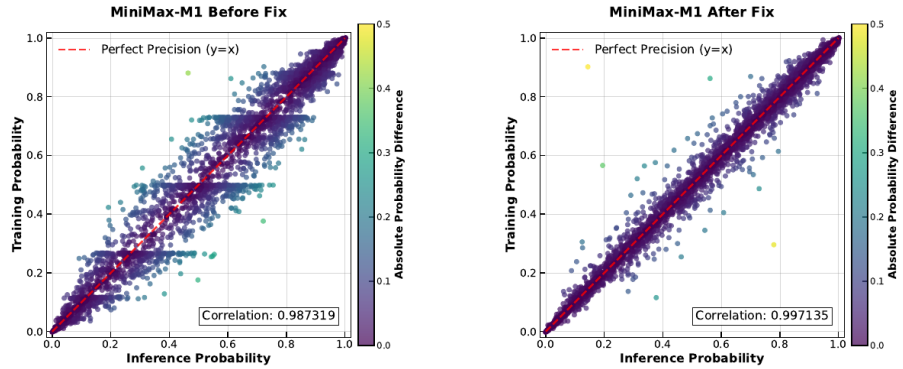
$$r_t = \frac{\pi_\theta(o_t)}{\pi_{\theta_{old}}(o_t)} \approx \frac{p_{train}(o_t)}{p_{infer}(o_t)}$$

분자와 분모를 서로 다른 프로그램이 계산 → 모델이 하나도 안 바뀌었는데도 r ≠ 1

token	p_infer	p_train	r (update 전)	ε = 0.2이면
흔한 token	0.40	0.42	1.05	정상
드문 token (예: Wait)	0.010	0.014	1.40	clip → gradient 0

설명용 예시 숫자. 드문 token일수록 상대 오차가 커서 더 크게 틀어짐

(3) 결과: 변한 게 없는 token이 잘못 clip되거나, weight가 엉뚱해져 gradient 방향이 noise로 흔들림 → M1에서 reward가 오르지 않음



M1 Fig.3 — 가로: p_infer, 세로: p_train. 좌: fix 전 (corr 0.987) / 우: LM head FP32 후 (0.997)

(4) 해결: LM head만 FP32

- 층별 분석 → 오차 주범은 LM head의 high-magnitude activation
- LM head만 FP32로 계산 → corr **0.987 → 0.997**, 학습 내내 유지
- 마지막 층 하나만 바꾸므로 추가 비용 작음
- ScaleRL에서 재현: asymptote 0.52 → 0.61

같은 절의 다른 안정화 trick

- AdamW: gradient 대부분 1e-14 이하 → eps 1e-15, β2 0.95
- 반복 생성 조기 종료: 3,000 token 연속 prob > 0.99면 중단

③ CISPO (1) 문제 제기: Reflection token은 어떻게 학습되나

논문의 관찰 (MiniMax-M1 §3.1)

- M1 zero-RL에서 GRPO가 long CoT의 등장을 방해 → ablation 결과 원인은 PPO/GRPO의 clipping
- Reflection token: Wait, However, Recheck, Aha — 추론을 재검토하거나 새 경로로 전환하는 "fork"
- base model에선 드물게 나와서 **확률이 매우 낮음**
- 이런 low-prob token이 entropy 안정과 scalable RL에 중요 (Cui et al., 2025; Wang et al., 2025)

예: reflection이 오류를 고친다

```
15 × 12 = 170
Wait, let me recheck.
15 × 10 = 150
15 × 2 = 30
Therefore, 180.
```

Wait가 기존 추론을 재검토하는 계기가 되어 정답으로 이어짐

Wait의 확률은 왜 올라갈까?

응답 A

```
17 × 13 = 211.
Answer: 211.
```

Reward = 0

응답 B

```
17 × 13 = 211.
Wait, let me recheck.
17 × 3 = 51
170 + 51 = 221
Answer: 221.
```

Reward = 1

- GRPO는 같은 질문의 응답들끼리 reward를 비교 → 응답 B의 advantage > 0
- → B에 포함된 **모든 token**의 확률을 올림 (Wait 포함)
- Wait에 따로 reward를 주는 게 아님: 응답 전체가 같은 신호를 받음 (credit assignment)
- 그런데 Wait는 원래 확률이 매우 낮음 → 조금만 올라도 ratio가 크게 튐 → clipping에 걸림 (다음 장)

③ CISPO (2) GRPO의 Clipping은 어떻게 작동하나

$$r_t(\theta) = \frac{\pi_\theta(o_t \mid q, o_{<t})}{\pi_{\theta_{old}}(o_t \mid q, o_{<t})}$$

현재 모델(학습 중)의 token 확률

rollout을 만든 이전 모델의 token 확률

$$L_t(\theta) = \min\left(r_t(\theta)\, A_t \,,\ \mathrm{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, A_t \right)$$

둘 중 작은 값 (보수적 update)

ratio를 [1-ε, 1+ε]로 자름
ε: 허용 변화 범위 (보통 0.2)

원래 objective
ratio × advantage

advantage
좋은 응답 +, 나쁜 응답 -

예: **A_t = 1** (좋은 응답의 token), **ε = 0.2** → 상한 1.2. 확률을 올려가면?

이전 확률	현재 확률	Ratio	Clipped objective	Gradient
10%	10%	1.0	1.0	있음
10%	11%	1.1	1.1	있음
10%	12%	1.2	1.2	있음
10%	15%	1.5	1.2	0
10%	20%	2.0	1.2	0

$$r_t > 1.2: \quad L_t(\theta) = \min(r_t,\, 1.2) \cdot 1 = 1.2 \ \Rightarrow\ \frac{\partial L_t}{\partial \theta} = 0$$

A_t < 0이면 반대로 하한 0.8 아래에서 0 · 이 token의 clipped 항 gradient만 0 (모델 전체 gradient가 0인 건 아님)

왜 reflection token에서 문제인가

	Therefore	Wait
이전 확률	50%	1%
현재 확률	60%	3%
절대 증가량	+10%p	+2%p
Ratio	1.2	3.0

분모(이전 확률)가 작으면 작은 변화도 큰 ratio → 상한 1.2를 쉽게 넘음

한 rollout batch를 여러 번 update하는 동안

단계	Old	Current	Ratio	Gradient (A>0)
Rollout 생성	0.01	0.01	1.0	있음
Update #1	0.01	0.011	1.1	있음
Update #2	0.01	0.014	1.4	0
Update #3	0.01	0.020	2.0	0

Old policy는 고정, current만 계속 변함 → ratio가 1.2를 넘는 순간부터 Wait의 양의 학습 신호 차단. M1은 batch당 **16번** off-policy update → 특히 두드러짐

③ CISPO (3) Objective 해부: Equation (4)와 Advantage

MiniMax-M1 Eq.(4): REINFORCE에 잘린(clipped) IS weight를 곱한 형태

old policy로 질문당 응답 G개 생성 / 모든 응답의 모든 token / advantage (응답 단위)

$$\mathcal{J}_{CISPO}(\theta) = \mathbb{E}_{q,\,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}}\left[\frac{1}{\sum_i |o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\mathrm{sg}(\hat{r}_{i,t}(\theta))\,\hat{A}_{i,t}\,\log\pi_\theta(o_{i,t}\mid q, o_{i,<t})\right]$$

전체 token 수로 나눔 (token-level 평균) / clip된 IS weight, sg = 상수 취급 / gradient는 여기로만 흐름

원래 IS ratio / 상한만 tune

$$\hat{r}_{i,t}(\theta) = \mathrm{clip}\left(r_{i,t}(\theta),\, 1-\epsilon_{low}^{IS},\, 1+\epsilon_{high}^{IS}\right)$$

하한: ε_low를 크게 → 사실상 없음

$$\hat{A}_{i,t} = \frac{R_i - \mathrm{mean}(\{R_j\}_{j=1}^{G})}{\mathrm{std}(\{R_j\}_{j=1}^{G})}$$

응답 i의 reward - 같은 질문 응답들의 평균

같은 질문 응답들의 표준편차

같은 응답의 모든 token이 **같은 Â를 공유** → Wait만 따로 점수를 받지 않음

풀이: 질문 하나에 응답 4개

응답	Reward	Advantage	학습 방향
A	1	+1	확률 ↑
B	1	+1	확률 ↑
C	0	−1	확률 ↓
D	0	−1	확률 ↓

$\mu = \frac{1+1+0+0}{4} = 0.5, \quad \sigma = 0.5$

$\hat{A}_A = \frac{1-0.5}{0.5} = +1, \quad \hat{A}_C = \frac{0-0.5}{0.5} = -1$

③ CISPO (4) Stop-gradient: GRPO와 Gradient 직접 비교

Stop-gradient (sg)란?

Forward에서는 값을 그대로 쓰고, Backward에서는 상수로 취급

$$y = x \cdot x \;\Rightarrow\; \frac{dy}{dx} = 2x$$

$$y = \mathrm{sg}(x) \cdot x \;\Rightarrow\; \frac{dy}{dx} = \mathrm{sg}(x)$$

계산 (x = 3)	Forward	Gradient
x · x	9	6
sg(x) · x	9	3

Forward 결과는 같고 **gradient 경로만** 다름 → CISPO에서 IS weight는 곱해지는 상수가 되고, gradient는 log π로만 흐름

```
w = ratio.clamp(max=eps_high).detach()  # sg
loss = -(w * adv * logp).mean()
# ms-swift: --loss_type cispo --epsilon_high 5.0
```

같은 token, 두 objective (A = 1, r = 3, ε = 0.2)

GRPO: objective를 clip

$$J_{GRPO} = \min(rA,\ \mathrm{clip}(r, 0.8, 1.2)A) = \min(3,\ 1.2) = 1.2$$

$$\nabla_\theta J_{GRPO} = 0$$ → 양의 학습 신호 차단

CISPO: IS weight만 clip (잘린 weight = min(3, 1.2) = 1.2)

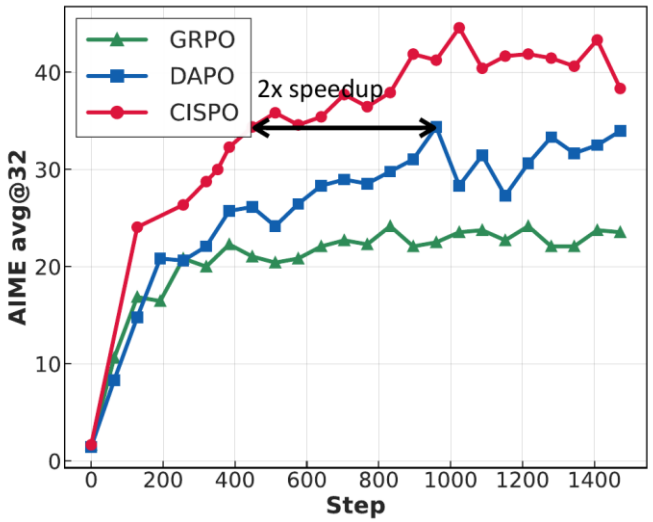
$$J_{CISPO} = \mathrm{sg}(\hat{r})\, A \log \pi_\theta(o_t) = 1.2 \log \pi_\theta(o_t)$$

$$\nabla_\theta J_{CISPO} = 1.2 \nabla_\theta \log \pi_\theta(o_t)$$ → 제한된 크기로 학습 지속

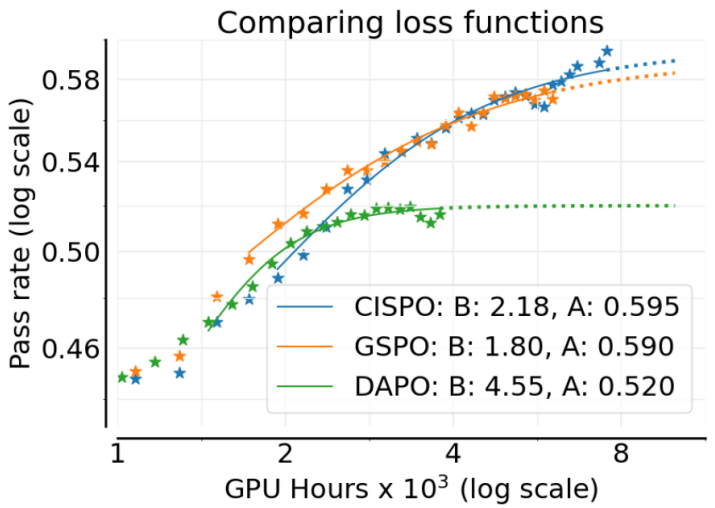
	GRPO	CISPO
적용 방식	Objective clipping	IS weight clipping
Objective 값	1.2 (상수)	1.2 · log π
Gradient	0	1.2 ∇log π
결과	양의 학습 신호 차단	제한된 크기로 학습 지속

일반화: PPO/GRPO의 trust region = 이런 token을 지우는 mask M, CISPO = M ≡ 1 (논문 Eq.6–7)

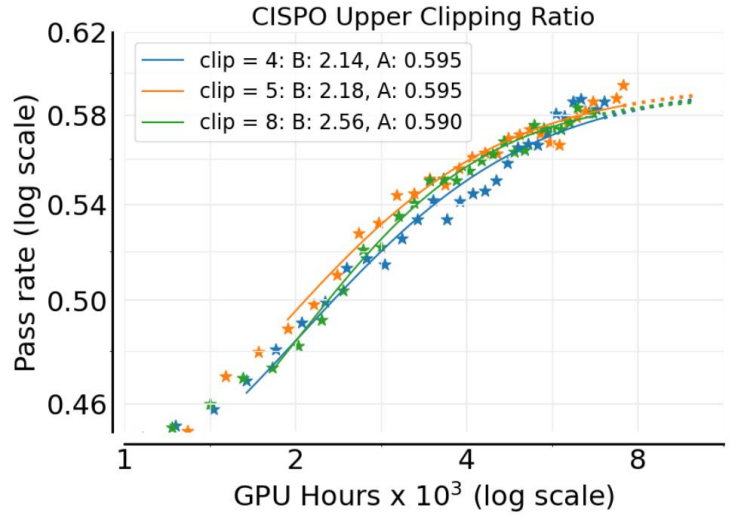
③ CISPO (5) Evidence



MiniMax-M1 Fig.2 — Qwen2.5-32B zero-RL, AIME24: CISPO가 DAPO 성능에 50% step으로 도달 (2× speedup)



ScaleRL Fig.5a — asymptote A: DAPO 0.52 · GSPO 0.59 · CISPO 0.595 → 최종 recipe에 CISPO 채택



ScaleRL Fig.19b — CISPO ε_max ∈ {4, 5, 8}: 거의 차이 없음 (DAPO는 ε_max가 asymptote 자체를 바꿈)

	clip 대상	clip된 token의 gradient	주요 hparam	ε 민감도
GRPO	token ratio r (min / clip)	0 (drop)	ε = 0.2	높음
DAPO	token ratio r, 비대칭 (clip-higher)	0 (범위만 넓힘)	ε_low 0.2 / ε_high 0.28	높음 (asymptote A 변화)
CISPO	IS weight (sg, upper only)	유지 (크기만 cap)	ε_max ≈ 4–8	낮음

Takeaway: token을 버리지 말고 weight를 잘라라 → 효율 ↑ · hparam robust ↑ · GSPO보다 안정 (ScaleRL에서 GSPO는 학습 중 divergence)

Review Summary: 강점 · 한계 · 이후 영향

① **Lightning attention** · 긴 생성을 싸게 → ② **FP32 LM head** · 불일치 해결 → ③ **CISPO** · token을 안 버리는 RL

강점

- 구조(①) · infra(②) · 알고리즘(③)을 하나의 RL scaling 문제로 엮은 end-to-end 보고서
- 1M context open-weight reasoning model, 100K 생성에서 FLOPs ~25% (vs R1)
- 전체 RL ≈ $0.53M (512 × H800, 3주) — 재현 가능한 비용 공개

한계 · 질문

- CISPO 비교는 Qwen2.5-32B zero-RL · AIME24 **단일 설정**
- truncated IS → biased gradient, lower bound 없음, 논문에 ε 값 미기재
- 7 : 1 hybrid의 retrieval trade-off는 M1 논문 자체에선 거의 다루지 않음

이후 영향

- ScaleRL이 CISPO를 best loss로, FP32 LM head를 recipe에 채택
- train–inference mismatch가 RL infra의 표준 점검 항목이 됨
- Lambert: "1년 뒤에도 살아남은 report", 2026에도 CISPO relevance ↑

한 줄 평: reasoning을 길게 하려면 구조 · 수치 정밀도 · 알고리즘을 같이 바꿔야 한다는 걸 보여준 report. 그중 FP32 LM head와 CISPO는 이후 RL recipe의 표준이 됨

Open Questions & Take-Home Message

Open questions

- RL은 새 capability를 발견하나, pretraining이 배운 걸 끌어내는(elicit) 것인가? — Lambert: RL은 새로운 걸 가르친다
- 더 좋은 pretraining data 없이 reasoning training은 어디까지? → pretraining은 죽지 않았다
- Agentic RL (tool use + reasoning)은 근본적으로 다른 recipe가 필요할까?
- RL scaling을 체계적으로 연구할 수 있을까? (ScaleRL, Khatri et al., 2025)
- 1 · 2 · 5 · 10년 뒤 pretraining vs RL compute 비율은?

Take-Home Message

- RLVR = 정답을 아는 문제로 RL을 scale → inference-time scaling을 unlock
- 2025: R1 recipe가 1년 가까이 default → 연말 **multi-teacher on-policy distillation**으로 이동
- frontier는 math/code reasoning에서 tool use · agent로 빠르게 이동
- Recipe 공통 원리: gradient가 사는 곳에 compute를 쓰고 (difficulty filtering), 모든 token의 신호를 살린다 (token-level loss, CISPO)
- lasting trick은 noise 속에서 늦게 보인다 → 남의 recipe를 가정하지 말고 직접 실험

RL libraries: slime · SkyRL · veRL · OpenRLHF · TRL · Open Instruct